

Experiment 08 – Views and Materialized Views

Sahil Goyal | UID: 24BDA70148

Aim

To study and implement database views and materialized views in PostgreSQL, and to apply GRANT and REVOKE to control access to a view.

Objective

- Build a complex view that joins multiple tables and derives a calculated column.
- Create a database user and control its access to a view using GRANT and REVOKE.
- Build a materialized view that pre-computes an aggregated category-wise report.
- Use conditional aggregation (CASE) to classify each group's performance.

Part A (i): Complex View

A retailer keeps three tables — registered customers, a product catalog, and the sales orders placed against them. Build a view that reports, for every order, the customer's name, the quantity ordered, and the final billed amount after the order's discount is applied.

Schema and sample data

```
CREATE TABLE customer_master (  
    customer_id VARCHAR(5) PRIMARY KEY,  
    full_name   VARCHAR(50) NOT NULL,  
    city        VARCHAR(30)  
);  
  
CREATE TABLE product_catalog (  
    product_id   VARCHAR(5) PRIMARY KEY,  
    product_name VARCHAR(50) NOT NULL,  
    unit_price   NUMERIC(10,2) NOT NULL  
);  
  
CREATE TABLE sales_orders (  
    order_id          SERIAL PRIMARY KEY,  
    product_id        VARCHAR(5) REFERENCES product_catalog(product_id),  
    quantity          INT NOT NULL,  
    customer_id       VARCHAR(5) REFERENCES customer_master(customer_id),  
    discount_percent   NUMERIC(5,2),  
    order_date        DATE NOT NULL  
);  
  
INSERT INTO customer_master (customer_id, full_name, city) VALUES  
( 'C1', 'Amit Sharma', 'Delhi'),  
( 'C2', 'Priya Verma', 'Mumbai'),  
( 'C3', 'Ravi Kumar', 'Bangalore'),  
( 'C4', 'Neha Singh', 'Kolkata'),  
( 'C5', 'Arjun Mehta', 'Hyderabad');  
  
INSERT INTO product_catalog (product_id, product_name, unit_price) VALUES  
( 'P1', 'Smartphone X100', 25000),  
( 'P2', 'Laptop Pro 15', 65000),  
( 'P3', 'Wireless Earbuds', 5000),  
( 'P4', 'Smartwatch Fit', 30000),  
( 'P5', 'Gaming Console', 45000);  
  
INSERT INTO sales_orders
```

```
(product_id, quantity, customer_id, discount_percent, order_date) VALUES
('P1', 2, 'C1', 5, '2025-09-01'),
('P2', 1, 'C2', 10, '2025-09-02'),
('P3', 3, 'C3', 0, '2025-09-03'),
('P4', 1, 'C4', 8, '2025-09-04'),
('P5', 1, 'C5', 12, '2025-09-05'),
('P1', 1, 'C2', 5, '2025-09-06'),
('P3', 2, 'C1', 0, '2025-09-07');
```

View definition

The worksheet's final amount only computed the discount value ($\text{price} \times \text{qty} \times \text{discount\%} / 100$), not the amount actually billed. Corrected below to subtract the discount from the gross total.

```
CREATE OR REPLACE VIEW vw_sales_report AS
SELECT
    t.order_id,
    cc.full_name,
    t.quantity,
    t.final_amount
FROM (
    SELECT
        so.order_id, so.product_id, so.customer_id, so.quantity,
        (pc.unit_price * so.quantity)
        - (pc.unit_price * so.quantity * so.discount_percent / 100)
        AS final_amount
    FROM sales_orders so
    JOIN product_catalog pc ON so.product_id = pc.product_id
) t
JOIN customer_master cc ON cc.customer_id = t.customer_id;

SELECT * FROM vw_sales_report;
```

Output

Unsaved e

```

7 FROM (
8     SELECT
9         (pc.unit_price * so.quantity)
10        - (pc.unit_price * so.quantity * so.discount_percent / 100)
11        AS final_amount
12    FROM sales_orders so
13    JOIN product_catalog pc ON so.product_id = pc.product_id
14 ) t
15 JOIN customer_master cc ON cc.customer_id = t.customer_id;
16
17 SELECT * FROM vw_sales_report;
18
19

```

Results Chart

order_id	full_name	quantity	final_amount
1	Amit Sharma	2	47500.000000000000000000
2	Priya Verma	1	58500.000000000000000000
3	Ravi Kumar	3	15000.000000000000000000
4	Neha Singh	1	27600.000000000000000000
5	Arjun Mehta	1	39600.000000000000000000

Grant and revoke access on the view

```
CREATE USER exp_8_user WITH PASSWORD '1234';
GRANT SELECT ON vw_sales_report TO exp_8_user;

-- to withdraw the access later:
REVOKE SELECT ON vw_sales_report FROM exp_8_user;
```

Part A (ii): Materialized View for Category Report

A separate schema tracks product categories, products, customers, and individual order line items. Build a materialized view that summarizes, per category, the total units sold, number of orders, number of unique customers, total revenue, average revenue per order, and a performance label.

Schema and sample data

```
DROP TABLE IF EXISTS order_details CASCADE;
DROP TABLE IF EXISTS product_master CASCADE;
DROP TABLE IF EXISTS category_master CASCADE;
DROP TABLE IF EXISTS customer_master CASCADE;

CREATE TABLE category_master (
    category_id VARCHAR(5) PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL
);

CREATE TABLE product_master (
    product_id VARCHAR(5) PRIMARY KEY,
    product_name VARCHAR(50) NOT NULL,
    category_id VARCHAR(5) REFERENCES category_master(category_id)
);

CREATE TABLE customer_master (
    customer_id VARCHAR(5) PRIMARY KEY,
    customer_name VARCHAR(50) NOT NULL
);

CREATE TABLE order_details (
    order_id VARCHAR(5) NOT NULL,
    customer_id VARCHAR(5) REFERENCES customer_master(customer_id),
    product_id VARCHAR(5) REFERENCES product_master(product_id),
    quantity INT NOT NULL,
    unit_price NUMERIC(10,2) NOT NULL
);

INSERT INTO category_master (category_id, category_name) VALUES
('C1', 'Electronics'),
('C2', 'Clothing'),
('C3', 'Books');

INSERT INTO product_master (product_id, product_name, category_id) VALUES
('P101', 'Laptop', 'C1'),
('P102', 'Headphones', 'C1'),
('P201', 'Jacket', 'C2'),
('P202', 'Shoes', 'C2'),
('P301', 'SQL Book', 'C3');

INSERT INTO customer_master (customer_id, customer_name) VALUES
('U1', 'Aman'),
('U2', 'Priya');
```

```

('U3', 'Rahul'),
('U4', 'Simran');

INSERT INTO order_details
(order_id, customer_id, product_id, quantity, unit_price) VALUES
('O101', 'U1', 'P101', 1, 5000),
('O102', 'U2', 'P102', 2, 1000),
('O103', 'U1', 'P201', 1, 3000),
('O104', 'U3', 'P202', 2, 2000),
('O105', 'U4', 'P301', 3, 500),
('O106', 'U2', 'P101', 1, 5000),
('O107', 'U3', 'P201', 1, 3000),
('O108', 'U4', 'P102', 1, 1000);

```

Materialized view definition

```

DROP MATERIALIZED VIEW IF EXISTS mw_category_report;

CREATE MATERIALIZED VIEW mw_category_report AS
SELECT
    cm.category_id    AS cat_id,
    cm.category_name AS cat_name,
    SUM(od.quantity)   AS total_product,
    COUNT(DISTINCT od.order_id) AS total_orders,
    COUNT(DISTINCT od.customer_id) AS unique_customer,
    SUM(od.quantity * od.unit_price) AS total_revenue,
    ROUND(
        SUM(od.quantity * od.unit_price)
        / COUNT(DISTINCT od.order_id), 2
    ) AS avg_revenue,
    CASE
        WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
        WHEN SUM(od.quantity * od.unit_price) >= 6000  THEN 'Good'
        WHEN SUM(od.quantity * od.unit_price) >= 3000  THEN 'Average'
        ELSE 'Low'
    END AS category_performance
FROM order_details od
JOIN product_master pm ON od.product_id = pm.product_id
JOIN category_master cm ON cm.category_id = pm.category_id
GROUP BY cm.category_id, cm.category_name;

SELECT * FROM mw_category_report;

```

Output

```

4  SELECT
11      ROUND(
12          SUM(od.quantity * od.unit_price)
14      ) AS avg_revenue,
15      CASE
16          WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
17          WHEN SUM(od.quantity * od.unit_price) >= 6000 THEN 'Good'
18          WHEN SUM(od.quantity * od.unit_price) >= 3000 THEN 'Average'
19          ELSE 'Low'
20      END AS category_performance
21 FROM order_details od
22 JOIN product_master pm ON od.product_id = pm.product_id
23 JOIN category_master cm ON cm.category_id = pm.category_id
24 GROUP BY cm.category_id, cm.category_name;
25
26 SELECT * FROM mw_category_report;
27

```

Results Chart

cat_id	cat_name	total_product	total_orders	unique_customer	total_revenue	avg_revenue	category_performance
C1	Electronics	5	4	3	13000.00	3250.00	Excellent
C2	Clothing	4	3	2	10000.00	3333.33	Excellent
C3	Books	3	1	1	1500.00	1500.00	Low

Learning Outcome

1. Understood the concept and implementation of database views and materialized views.
2. Used aggregate functions such as SUM(), COUNT(), and DISTINCT to build a summarized report.
3. Applied joins and conditional aggregation (CASE) across multiple related tables.
4. Used GRANT and REVOKE to control a user's access to a view.